

Abaqus Handbuch-Chatbot

Ein Retrieval-Augmented-Generation-System zur Beantwortung von Bedienungsfragen
zur FEA-Software Abaqus

Florian Heithoff

Matrikelnummer: 30483505

NLP-Studienmodul

Fachhochschule Südwestfalen

Betreuer: Prof. Dr. C. Gawron

03. September 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Über Abaqus	3
1.2	Motivation und Zielsetzung	3
2	Vorgehen	3
2.1	Methodisches Vorgehen	3
2.2	Design-Prinzip: isolierte Ein-Dimensions-Vergleiche	3
2.3	Demo-Korpus vs. Produktivkorpus	4
3	Das RAG-System: Architektur	4
3.1	Systemüberblick	4
3.2	Zentrale Designentscheidungen	4
3.3	Untersuchte Pipeline-Varianten	5
4	Implementierung	5
4.1	Projektstruktur	5
4.2	Zentrale Code-Bausteine	5
4.2.1	Loader	6
4.2.2	Chunker	6
4.2.3	Vectorstore	6
4.2.4	Prompts	6
4.2.5	LCEL-Kette	6
4.2.6	Source-Tracking	7
4.3	Austauschbare Komponenten (Registry-Pattern)	7
4.4	Tests	7

5	Untersuchte Komponenten	7
5.1	PDF-Parser	8
5.2	Chunking-Strategien	8
5.3	Embedding-Modelle	8
5.4	Retrieval-Strategien	8
5.5	Antwort-LLMs	9
6	Validierung: Quantitative Evaluation mit RAGAS	9
6.1	Warum RAGAS, mit festem, unabhängigem Richter	9
6.2	Die fünf RAGAS-Metriken	9
6.3	Evaluations-Setup	10
6.4	Bewertung der einzelnen Komponenten	10
6.5	Best-of-Breed-Experiment	11
7	Bekannte Grenzen, Ausblick und Fazit	11
7.1	Bekannte Grenzen und Ausblick	11
7.2	Fazit	12
7.3	Hinweis zum Einsatz von KI-Werkzeugen	13

1 Einleitung

1.1 Über Abaqus

Abaqus ist eine Simulationssoftware aus der Strukturmechanik, die mithilfe der Finite-Elemente-Methode (FEM) strukturmechanische Problemstellungen numerisch löst. Eingesetzt wird sie vor allem in der Konstruktion und Entwicklung von mechanisch belasteten Bauteilen. Sie erlaubt es, deren Verhalten unter Last bereits am digitalen Modell zu prüfen, bevor ein physischer Prototyp gebaut wird. Abaqus ist eine komplexe Software mit zahlreichen Modulen, Optionen und einer umfangreichen Keyword-/Input-Deck-Syntax (siehe [Abaqus2017_KEYWORDS.pdf](#)), deren sichere Bedienung entsprechend anspruchsvoll ist.

1.2 Motivation und Zielsetzung

Die offiziellen Abaqus-Handbücher umfassen mehrere tausend Seiten, verteilt auf mehrere PDF-Dokumente. Eine reine Volltextsuche im PDF-Reader findet zwar Fundstellen, liefert aber keine kontextuelle Antwort auf eine konkrete Bedienungsfrage. Ziel dieser Arbeit ist daher ein Chatbot, der Fragen in natürlicher Sprache beantwortet, und zwar ausschließlich auf Basis der offiziellen Handbücher und nicht aus dem "Weltwissen" des zugrunde liegenden Sprachmodells. Zwei Eigenschaften sind für die Antwortqualität zentral: Eine seitengenaue Quellenangabe, die die Antwort im Original nachprüfbar macht, und ein Gesprächsgedächtnis, das Folgefragen versteht. Halluzinationen wird dabei durch einen strikten System-Prompt entgegengewirkt, sobald der abgerufene Kontext die Frage nicht abdeckt.

Über diese Grundfunktion hinaus ist jede Komponente der Verarbeitungspipeline – Parser, Chunking, Embedding, Retrieval und Antwort-LLM – bewusst austauschbar gestaltet. Das erlaubt es, für jede Komponente empirisch zu prüfen, welche Variante die beste Antwortqualität liefert.

2 Vorgehen

2.1 Methodisches Vorgehen

Das Projekt folgt einem iterativen Vorgehen in drei Phasen. Zunächst wird eine funktionierende Produktiv-Baseline mit allen fünf Pipeline-Stufen – Parser, Chunking, Embedding, Retrieval und Antwort-LLM – aufgebaut. Darauf aufbauend wird systematisch je eine Pipeline-Dimension gegen diese Baseline variiert und die Auswirkung quantitativ per RAGAS gemessen. Abschließend werden die jeweiligen Einzelsieger im Best-of-Breed-Experiment zusammengeführt. Jede Designentscheidung ist dadurch nicht nur begründet, sondern nachträglich auch empirisch geprüft.

2.2 Design-Prinzip: isolierte Ein-Dimensions-Vergleiche

Jede der sechs Vergleichsstudien variiert genau eine Dimension, während alle übrigen Pipeline-Stufen auf dem jeweiligen Produktiv-Default fixiert bleiben. Ziel ist es, gemessene Effekte einzelnen Ursachen zurechenbar zu halten, statt sie in einer vollen Kombinatorik zu vermischen, da mit

jeweils vier bis sieben Varianten eine vollständige Kombinatorik zu aufwändig wäre. Wechselwirkungen zwischen mehreren gleichzeitig veränderten Dimensionen werden dadurch nicht erfasst und werden erst im abschließenden Best-of-Breed-Experiment (Abschnitt 6.5) sichtbar.

2.3 Demo-Korpus vs. Produktivkorpus

Der Produktivkorpus umfasst vier vollständige Abaqus-Handbücher mit ca. 5.100 Seiten und 15.408 Chunks. Alle sechs Vergleichsstudien laufen jedoch nicht auf diesem vollen Korpus, sondern auf einem festen, deterministisch zusammengestellten Demo-Korpus von ca. 53 Seiten. Der kleinere Korpus wurde gewählt, da Unstructured (`hi_res`) auf dem vollen Korpus mit geschätzt mehreren Stunden Laufzeit nicht praktikabel wäre – Docling ist mit warmem Modell-Cache zwar ebenfalls deutlich langsamer als PyMuPDF4LLM, mit geschätzt rund einer Stunde für den vollen Korpus aber grundsätzlich machbar (siehe `docs/PARSER.md`). Zusätzlich wären auch RAGAS-Läufe mit LLM-Richter über mehrere Kombinationen hinweg auf dem vollen Korpus zeit- und kostenintensiv. Um trotz der reduzierten Größe aussagekräftig zu bleiben, wurde der Demo-Korpus gezielt aus genau den Inhaltstypen zusammengesetzt, die sich zuvor als unterschiedlich schwierig erwiesen hatten – darunter TOC-Tabellen, linienlose Mini-Tabellen, ein vollständiges Tutorial-Kapitel sowie ein Analyse-Kapitel.

3 Das RAG-System: Architektur

3.1 Systemüberblick

Das System besteht aus zwei vollständig entkoppelten Phasen. In der **Ingestion** – einmalig ausgeführt bzw. wiederholt bei geänderten PDFs – durchlaufen die Handbücher eine feste Kette: Ein Parser wandelt die PDFs in Text um, ein Chunker zerlegt diesen Text in handhabbare Abschnitte, ein Embedding-Modell wandelt jeden Abschnitt in einen Vektor um, und das Ergebnis landet in einem persistenten Chroma-Vectorstore. In der **Chat-Session** greift der Bot ausschließlich lesend auf diesen fertigen Vectorstore zu – ohne Rückwirkung auf die Wissensbasis während des Chats. Eine Nutzerfrage durchläuft dabei zunächst einen Contextualize-Step, der Folgefragen anhand des Gesprächsverlaufs eigenständig verständlich macht, wird anschließend vom Retriever gegen den Vectorstore aufgelöst und zusammen mit dem gefundenen Kontext an eine Answer-Chain übergeben, die aus System-Prompt, Kontext und Frage die endgültige Antwort samt Quellenangaben generiert.

3.2 Zentrale Designentscheidungen

Neben den fünf austauschbaren Pipeline-Stufen (Abschnitt 3.3) enthält das System eine Reihe infrastruktureller Entscheidungen, die bewusst *nicht* zum Gegenstand eines Vergleichs gemacht wurden, weil sie den Kern der RAG-Pipeline selbst nicht betreffen, sondern deren technisches Fundament bilden. Sie sind für alle in Abschnitt 5 untersuchten Varianten gleichermaßen gültig und werden in keiner der sechs Vergleichsstudien verändert:

- **Vektor-DB:** Chroma (lokal, persistent, SQLite-basiert) – kein Serverbetrieb nötig, native LangChain-Integration, Metadaten-Filterung

- **RAG-Orchestrierung:** LCEL-Runnables statt `langchain.chains` (in LangChain ≥ 1.0 entfernt).
- **Conversation Memory:** einfache Nachrichtenliste innerhalb der CLI-Session

3.3 Untersuchte Pipeline-Varianten

Im Gegensatz dazu wurden genau die fünf Stufen, die den eigentlichen Verarbeitungsweg von der PDF-Seite bis zur fertigen Antwort ausmachen, bewusst austauschbar gestaltet: Für jede von ihnen existiert im Code eine Registry mehrerer gleichwertiger Implementierungen (Abschnitt 4), sodass sich jede Stufe unabhängig von den übrigen gegen Alternativen testen lässt. Diese fünf Stufen sind zugleich die fünf Dimensionen der Vergleichsstudien aus Abschnitt 5, wo die jeweils getesteten Varianten und der eingesetzte Produktiv-Default im Detail aufgeführt sind. An dieser Stelle nur die Übersicht, wie viele Varianten je Dimension untersucht wurden:

- PDF-Parser – 4 getestete Varianten
- Chunking-Strategie – 6 getestete Varianten
- Embedding-Modell – 7 getestete Varianten
- Retrieval-Strategie – 4 getestete Varianten
- Antwort-LLM – 3 getestete Varianten

4 Implementierung

4.1 Projektstruktur

Der Kern der Anwendung ist als eigenständiges Python-Paket `src/rag_manual_bot/` organisiert und darin in vier funktionale Bereiche unterteilt. `scripts/`, `tests/`, `app.py` und `main.py` sind nicht Teil dieses Pakets, sondern liegen als eigenständige Verzeichnisse bzw. Dateien direkt im Projekt-Root. Sie importieren das Paket und nutzen dessen Funktionalitäten.

```
Abaqus_chatbot/
|-- src/rag_manual_bot/ <- das Python-Paket
| |-- ingestion/ Loader, Chunker, Parser-/Chunking-/Embedding-Backends
| |-- rag/ Vectorstore, Retriever, Retrieval-Backends, LCEL-Kette
| |-- eval/ RAGAS-Anbindung, Evaluations-Datensatz, Metriken
| |-- citations.py Quellen-Links
| '-- cli/chat.py Terminal-Chat-Oberflaeche
|
|-- scripts/ CLI-Entry-Points fuer Ingestion, Demo-Korpus-Aufbau, RAGAS-Laeufe
|-- tests/ 51 offline lauffaehige Tests
|-- app.py Frontend-Entry-Point
'-- main.py Terminal-Entry-Point
```

4.2 Zentrale Code-Bausteine

Dieser Abschnitt beschreibt, wie die in Abschnitt 3 konzeptionell vorgestellten Komponenten konkret im Code umgesetzt sind – mit Fokus auf Implementierungsdetails, Funktionsnamen und technische Kniffe je Baustein.

4.2.1 Loader

Die Kernfunktion `load_pdf()` in `ingestion/loader.py` kombiniert zwei Bibliotheken: `pymupdf4llm` liefert den Markdown-Text pro Seite, `pymupdf` zusätzlich die im PDF eingebetteten Page-Labels, deren Auflösung über `doc[pdf_index].get_label()` erfolgt – mit Fallback auf die sequenzielle Position, falls kein Label vorhanden ist. Rückgabetyt ist eine Liste von `PageDocument`-Objekten (Text, Quelldatei, gedruckte Seitenzahl); leere Seiten werden bereits hier herausgefiltert, bevor sie in den Chunker gelangen.

4.2.2 Chunker

Die Kernfunktion `chunk_pages()` in `ingestion/chunker.py` ist nur ein dünner Wrapper um `ingestion/chunking_backends/header_recursive_backend.py` mit den Produktiv-Settings (`CHUNK_SIZE`, `CHUNK_OVERLAP`) – dieselbe Backend-Funktion ist zusätzlich unter zwei weiteren Größenvarianten sowie drei strukturell anderen Chunking-Strategien registriert (siehe Abschnitt 4 zum Registry-Pattern).

4.2.3 Vectorstore

Die Kernfunktionen `build_vectorstore()` und `load_vectorstore()` in `rag/vectorstore.py` persistieren lokal über Chroma (SQLite-basiert). Beide tragen einen optionalen `embeddings`-Parameter (eine fertige `Embeddings`-Instanz statt eines Modellnamens), da der Vectorstore zwingend mit demselben Embeddings-Objekt geladen werden muss, mit dem er aufgebaut wurde; `build_vectorstore()` löscht vor dem Neuaufbau alle bestehenden Einträge (`store.delete(ids=existing_ids)`) und fügt Dokumente in Batches von 200 ein.

4.2.4 Prompts

`rag/prompts.py` definiert zwei getrennte Prompt-Templates für die zwei Aufgaben der Kette: Der Contextualize-Prompt formuliert Folgefragen anhand des Verlaufs eigenständig um, ohne sie zu beantworten; der Answer-Prompt legt die Systemrolle fest ("technischer Assistent für Abaqus"), erzwingt Kontext-Bindung und ein explizites "Ich weiß es nicht" bei fehlender Abdeckung. Beide binden den bisherigen Gesprächsverlauf über `MessagesPlaceholder("chat_history")` als vollwertige Chat-Nachrichten ein, nicht als reinen Textblock.

4.2.5 LCEL-Kette

Die Kette wird in `rag/chain.py`, Funktion `build_rag_chain(vectorstore, llm_provider, retrieval_strategy)`, schrittweise über verkettete `RunnablePassthrough.assign(...)`-Aufrufe konstruiert; eine `RunnableBranch` überspringt dabei den Contextualize-Schritt bei leerem Gesprächsverlauf und spart so einen unnötigen LLM-Call. Jeder `.assign(...)`-Schritt reichert das durchlaufende Dict um ein weiteres Feld an (`standalone_question`, `source_documents`, `context`, `answer`). `llm_provider` und `retrieval_strategy` werden als Parameter durchgereicht und wählen aus den jeweiligen Registries (Abschnitt 4) – derselbe Kettenaufbau bedient damit sowohl den Produktivbetrieb als auch alle Vergleichsstudien.

4.2.6 Source-Tracking

`format_docs()` baut aus den abgerufenen Chunks den nummerierten Kontextblock für das LLM, inklusive Quelle und Seitenzahl als Zitat; `citations.py` erzeugt daraus eine klickbare `file://`-URL mit PDF-Seitenanker (`#page=N`). Der Anker muss sich auf die physische Seitenposition beziehen statt auf das gedruckte Label, da PDF-Viewer `#page=N` als reinen Dateiindex interpretieren – die Pipeline führt daher beide Werte parallel mit `_format_sources()` (in `cli/chat.py` bzw. `app.py`) dedupliziert die Quellen abschließend nach Datei und Seite, wobei die Reihenfolge der ersten Nennung erhalten bleibt.

4.3 Austauschbare Komponenten (Registry-Pattern)

Ein einheitliches Implementierungsprinzip zieht sich durch alle fünf austauschbaren Pipeline-Stufen: Für jede Stufe existiert eine Registry – ein Dictionary, das einen Namen auf eine Implementierung oder Factory abbildet. Neue Varianten werden als zusätzlicher Registry-Eintrag ergänzt, ohne den übrigen Code zu verändern; genau das ermöglicht denselben Code-Pfad sowohl für den Aufbau der Vergleichs-Collections als auch für die automatisierten RAGAS-Vergleiche. Konkret:

- `parser_backends/`: 4 Parser mit identischer Schnittstelle `parse(pdf_path) -> dict[int, str]`
- `chunking_backends/`: 6 Chunking-Strategien
- `embedding_models.py::EMBEDDING_BACKENDS`: 7 Embedding-Modelle (3 OpenAI-Factories, 4 lokale HuggingFace-Factories mit Lazy-Import)
- `retrieval_backends.py::RETRIEVAL_BACKENDS`: 4 Retrieval-Strategien
- `rag/llms.py::get_llm(provider)`: 3 Antwort-LLM-Provider (OpenAI, Mistral, GPT-4o)

4.4 Tests

Die Testsuite (51 Tests) läuft vollständig offline – es wird kein echter API-Call gegen OpenAI oder ein anderes Modell ausgeführt. Möglich macht das `tests/conftest.py`, das einen Dummy-`OPENAI_API_KEY` setzt, damit das beim Import validierende `Settings`-Modul auch ohne echten Key geladen werden kann, sowie eine selbstgeschriebene, deterministische `FakeEmbeddings`-Klasse anstelle echter Embedding-API-Aufrufe. Abgedeckt sind unter anderem die Chunker-Grundlogik, alle Chunking-Backends, die Embedding-Modell-Registry, die Retrieval-Backends, die LLM-Provider-Auswahl, die Custom-Demo-Kombinationslogik sowie der Eval-Datensatz. Ausführung über `python -m pytest tests/ -v`.

5 Untersuchte Komponenten

Für jede der fünf austauschbaren Pipeline-Stufen aus Abschnitt 3.3 wurden mehrere konkrete Implementierungen gegeneinander getestet. Dieses Kapitel listet je Dimension, *welche* Varianten untersucht wurden und markiert die jeweilige Produktiv-Default-Wahl; die zugehörige RAGAS-Methodik und die gemessenen Ergebnisse folgen in Kapitel 6.

5.1 PDF-Parser

Der Parser überführt jede Handbuchseite aus dem PDF in maschinenlesbaren Text.

- **PyMuPDF4LLM (Produktiv-Default)**: wandelt PDF-Seiten seitenweise in Markdown um, erhält dabei Überschriften, Tabellen und Seitenzahlen
- **Docling**: Dokumentenkonvertierungs-Toolkit mit Deep-Learning-basierter Layout- und Strukturerkennung
- **pdfplumber**: Python-Bibliothek mit Fokus auf präzise Extraktion von Text und Tabellen anhand der PDF-Koordinaten
- **Unstructured**: allgemeines Dokument-Partitionierungs-Framework, hier im rechenintensiven `hi_res`-Modus mit eigenem Layout-Erkennungsmodell getestet

5.2 Chunking-Strategien

Die Chunking-Strategie zerlegt den geparsten Text in einzelne, embedding-fähige Abschnitte.

- **Header+Recursive (Produktiv-Default)**: Hybrid aus Markdown-Header-Split und rekursivem Zeichen-Split als Fallback für überlange Abschnitte; getestet in drei Größenvarianten (400/800/1600 Zeichen)
- **Recursive-only**: rein zeichenbasierte rekursive Aufteilung ohne Berücksichtigung von Überschriften, als naive Baseline
- **Token-basiert**: Chunk-Grenzen anhand der Tokenzahl statt der Zeichenzahl
- **Semantic Chunking**: embedding-basierte Bestimmung der Chunk-Grenzen, setzt Trennstellen dort, wo sich die semantische Ähnlichkeit benachbarter Sätze am stärksten ändert

5.3 Embedding-Modelle

Das Embedding-Modell überführt jeden Chunk in einen Vektor und bildet damit die Grundlage der semantischen Suche.

- **OpenAI (API-basiert)**: `text-embedding-3-small` (Produktiv-Default), `text-embedding-3-large`, `text-embedding-ada-002`
- **Lokale HuggingFace-/sentence-transformers-Modelle**: `multilingual-e5-large`, `bge-m3`, `paraphrase-multilingual-mpnet`, `minilm-l6-en` – laufen lokal, ohne API-Kosten

5.4 Retrieval-Strategien

Die Retrieval-Strategie bestimmt, welche Chunks zur Anfragezeit aus dem Vectorstore ausgewählt werden.

- **MMR (Produktiv-Default)**: Max Marginal Relevance, wählt aus einem größeren Kandidatenpool eine thematisch diverse Teilmenge
- **Similarity**: reine Nächste-Nachbarn-Suche über Kosinus-Ähnlichkeit, ohne Diversitätsmechanismus

- **Cross-Encoder-Rerank**: sortiert die von der initialen Suche gelieferten Kandidaten mit einem Cross-Encoder-Modell neu
- **BM25-Hybrid**: kombiniert klassische BM25-Stichwortsuche mit dichter Vektorsuche über Reciprocal Rank Fusion

5.5 Antwort-LLMs

Das Antwort-LLM generiert aus Frage und abgerufenem Kontext die finale Antwort.

- **OpenAI gpt-4o-mini (Produktiv-Default)**
- **Mistral mistral-small-latest**
- **OpenAI gpt-4o**: volle Modellgröße derselben Generation wie der Produktiv-Default, läuft über denselben Hub/Key wie gpt-4o-mini

6 Validierung: Quantitative Evaluation mit RAGAS

6.1 Warum RAGAS, mit festem, unabhängigem Richter

Für diese Arbeit wird daher RAGAS eingesetzt, ein etabliertes Open-Source-Framework zur automatisierten, metrikbasierten Evaluation von RAG-Systemen. RAGAS benötigt für jede Frage vier Bestandteile: die Nutzerfrage selbst, die vom System generierte Antwort und die dafür abgerufenen Kontext-Chunks – für drei der fünf in Abschnitt 6 verwendeten Metriken zusätzlich eine manuell verfasste Referenzantwort ("Ground Truth"), gegen die Antwort und Kontext inhaltlich abgeglichen werden.

6.2 Die fünf RAGAS-Metriken

- **Faithfulness** – "Ist die Antwort durch den Kontext gedeckt?"
 - Braucht keine Referenz; zerlegt Antwort in atomare Claims, prüft jeden per NLI gegen den Kontext
 - Niedriger Wert = Modell halluziniert
- **AnswerRelevancy** – "Beantwortet die Antwort die gestellte Frage?"
 - Braucht keine Referenz; generiert Rückwärts-Fragen aus der Antwort, vergleicht Embedding-Ähnlichkeit zur Originalfrage
 - Niedriger Wert = Antwort ausweichend/zu allgemein
- **ContextPrecision** – "Sind die abgerufenen Chunks relevant?"
 - Braucht Referenzantwort; rangbasierte Average Precision über die Relevanz jedes abgerufenen Chunks
 - Niedriger Wert = Retrieval liefert Rauschen
- **ContextRecall** – "Deckt der Kontext alles ab, was nötig wäre?"

- Braucht Referenzantwort; prüft, ob jede Aussage der Referenzantwort im Kontext belegbar ist
- Niedriger Wert = Retrieval verpasst wichtige Information
- Misst ausschließlich Retrieval-Qualität, nicht das antwortende LLM – deshalb über verschiedene LLMs bei sonst gleicher Pipeline identisch (internes Konsistenz-Signal)
- **FactualCorrectness** – "Wie viele Fakten stimmen mit der Referenz überein?"
 - Braucht Referenzantwort; F1-Score über atomare Fakten (generierte Antwort vs. Referenz)
 - Durchgängig niedrigste Metrik (typisch 0,4–0,7) – normal bei striktem Fakten-F1 gegen manuell formulierte Referenzen, nicht direkt mit den anderen vier Metriken vergleichbar
- Alle Werte in $[0, 1]$, höher ist besser; der in dieser Abgabe verwendete $\emptyset$ -Score ist das ungewichtete Mittel aller fünf Metriken je Kombination (grobe Ranking-Kennzahl, für Detailanalysen sind Einzelmetriken aussagekräftiger)

6.3 Evaluations-Setup

- Fester 12-Fragen-Datensatz mit Referenzantworten (`eval/dataset.py`), deckt alle vier Handbuch-Themenbereiche ab
- Gleicher Demo-Korpus und gleiches Fragenset über alle sechs Studien – Vergleichbarkeit zwischen den Studien
- Die vier Pipeline-Stufen-Vergleiche (Parser, Chunking, Embedding, Retrieval) laufen mit fest auf `gpt-4o-mini` fixiertem Antwort-LLM und werden auch von `gpt-4o-mini` als Richter bewertet – da das Antwort-LLM dort für alle verglichenen Varianten konstant bleibt, ist ein Self-Preference-Bias ausgeschlossen; ergänzend wurden die Ergebnisse für Retrieval und Embedding stichprobenartig gegen ein zweites, größeres Richtermode (Mistral Large) validiert, ohne wesentliche Abweichungen zum primären Richter
- Der Antwort-LLM-Vergleich selbst (GPT-4o-mini, Mistral `mistral-small-latest`, GPT-4o) sowie das Best-of-Breed-Experiment testen gezielt mehrere Antwort-LLMs gegeneinander; da hier Modelle unterschiedlicher Anbieter direkt gegeneinander antreten, wird als Richter ein von allen Kandidaten unabhängiges Modell eingesetzt (Claude Sonnet 5, Anthropic)
- Gesamtumfang: $30 \text{ Kombinationen} \times 12 \text{ Fragen} \times 5 \text{ Metriken} = 1.800 \text{ Einzel-Scores}$
- Reproduzierbar über `scripts/evaluate_*_ragas.py`

6.4 Bewertung der einzelnen Komponenten

Mit der in den vorigen Abschnitten beschriebenen Methodik wurde jede der fünf in Kapitel 5 vorgestellten Dimensionen einzeln ausgewertet – jeweils mit allen übrigen Komponenten auf dem Produktiv-Default fixiert. Die vollständigen Ergebnistabellen, Diagramme und Rohdaten je Dimension sind nicht an dieser Stelle ausgeführt, sondern im interaktiven Dashboard

`docs/dashboard.html` zusammengefasst. Ergänzend liegt zu jeder Dimension eine eigene Dokumentationsdatei mit ausführlicher Diskussion vor: `docs/PARSER.md`, `docs/CHUNKING.md`, `docs/EMBEDDING.md`, `docs/RETRIEVAL.md` und `docs/LLM.md`. Im direkten Antwort-LLM-Vergleich unter dem unabhängigen Richter liegen GPT-4o-mini ($\varnothing$ 0,741) und GPT-4o ($\varnothing$ 0,739) praktisch gleichauf, mit Mistral ($\varnothing$ 0,662) deutlich dahinter. Abschließend wurde ein Best-of-Breed-Experiment durchgeführt, das die vier Einzelsieger kombiniert und prüft, ob sich deren Einzelverbesserungen tatsächlich addieren.

6.5 Best-of-Breed-Experiment

Das abschließende sechste Experiment kombiniert die vier Einzelsieger der vorigen Vergleiche zum Zeitpunkt ihrer Ermittlung (Unstructured + Semantic Chunking + `text-embedding-3-large` + Rerank) zu einer Gesamt-Pipeline und testet sie auf dem Demo-Korpus über die drei Antwort-LLMs (GPT-4o-mini, Mistral `mistral-small-latest`, GPT-4o) gegen die Produktiv-Baseline, bewertet durch den in Abschnitt 6 beschriebenen unabhängigen Richter. Im Durchschnitt über alle drei Antwort-LLMs und fünf Metriken schlägt Best-of-Breed die Baseline deutlich ($\varnothing$ 0,819 vs. 0,716, +14,4%). Der Vorsprung gilt jedoch nicht für jede Einzelmetrik und jedes Antwort-LLM gleichermaßen: Mit GPT-4o gewinnt Best-of-Breed alle fünf Metriken; mit Mistral sinkt die Faithfulness gegenüber der Baseline (0,799 auf 0,746); mit GPT-4o-mini sinken sowohl Faithfulness (0,906 auf 0,896) als auch FactualCorrectness (0,508 auf 0,479). Innerhalb von Best-of-Breed ist GPT-4o zugleich das mit Abstand stärkste Antwort-LLM ($\varnothing$ 0,863, vor GPT-4o-mini mit 0,821 und Mistral mit 0,772). Die methodische Kernaussage: Einzelverbesserungen addieren sich nicht automatisch zu einer gleichmäßigen Verbesserung – jede Kombination muss selbst gemessen werden. Details in `docs/dashboard.html`, Tab "Best-of-Breed".

7 Bekannte Grenzen, Ausblick und Fazit

7.1 Bekannte Grenzen und Ausblick

- Inkrementelles Ingestion-Update: Ein gezielter Fix ergänzt einen Datei-Fingerabdruck (SHA-256 je PDF, gespeichert in einem Manifest neben der Collection); seitdem verarbeitet jeder Ingestion-Lauf nur noch neue oder inhaltlich geänderte PDFs neu, entfernt gelöschte Dateien aus der Collection und überspringt unveränderte Dateien vollständig – ein Lauf ohne Änderungen ist damit ein reiner No-op ohne API-Kosten
- Seitenbasiertes Header-Splitting: Ein gezielter Fix führt Abschnitte, die ohne neue Überschrift über eine Seitengrenze laufen, inzwischen mit dem vorigen Abschnitt zusammen (die Seitenangabe wird dann zu einem Bereich); der PDF-Deep-Link zeigt bei solchen zusammengeführten Chunks aber weiterhin nur auf die Startseite, nicht auf eine ggf. relevantere Folgeseite
- Speicherumfang bewusst begrenzt: aktuell 4 von 21 verfügbaren Abaqus-2017-Handbüchern eingelesen ($\sim$ 5.100 von $\sim$ 21.800 Seiten), um Ingestion-Dauer und Kosten für Entwicklung/Tests gering zu halten

- Vergleichsumfang der vier Einzeldimensionen auf den Demo-Korpus beschränkt: Die Parser-, Chunking-, Embedding- und Retrieval-Vergleiche laufen ausschließlich auf dem ~ 53 -Seiten-Demo-Korpus, nicht auf dem vollen Produktivkorpus – wegen der Laufzeit von Unstructured (`hi_res`) und Semantic Chunking auf ~ 5.100 Seiten wurde eine Ausweitung im Rahmen dieser Abgabe bewusst nicht mehr durchgeführt. Das gilt auch für das Best-of-Breed-Experiment selbst, dessen Ergebnisse (Abschnitt 6.5) sich ebenfalls auf den Demo-Korpus beziehen

7.2 Fazit

Das im Rahmen dieser Arbeit entwickelte System zeigt, dass sich die mehrere tausend Seiten umfassenden Abaqus-Handbücher erfolgreich zu einem konversationsfähigen Chatbot verarbeiten lassen. Der Bot beantwortet Bedienungsfragen ausschließlich auf Basis der Handbuchinhalte, macht jede Antwort über eine seitengenaue Quellenangabe im Original nachprüfbar und versteht dank eines einfachen Gesprächsgedächtnisses auch Folgefragen.

Über die reine Funktionsfähigkeit hinaus war ein zentrales Ziel der Arbeit, die Designentscheidungen der Pipeline nicht nur zu begründen, sondern auch empirisch zu überprüfen. Dazu wurde jede der fünf austauschbaren Pipeline-Stufen – Parser, Chunking, Embedding, Retrieval und Antwort-LLM – über eine eigene Registry konsequent austauschbar gestaltet und in sechs unabhängigen RAGAS-Studien mit insgesamt 1.800 Einzel-Scores gegen Alternativen getestet. Damit liegen für alle fünf Dimensionen belastbare Tendenzaussagen vor, statt sich allein auf Intuition oder Einzelbeispiele zu verlassen.

Eine methodische Erkenntnis liefert dabei das abschließende Best-of-Breed-Experiment: Die Kombination der vier jeweils besten Einzelkomponenten schlägt die Produktiv-Baseline im Durchschnitt über alle drei getesteten Antwort-LLMs und alle fünf Metriken deutlich (+14,4 %), jedoch nicht bei jeder Einzelmetrik gleichermaßen – mit Mistral (`mistral-small-latest`) und, geringfügig, auch mit GPT-4o-mini als Antwort-LLM sinkt die Faithfulness gegenüber der Baseline leicht. Einzelverbesserungen addieren sich also nicht automatisch zu einer gleichmäßigen Verbesserung, und selbst ein neuer Gesamtsieger ersetzt nicht die Notwendigkeit, jede Kombination einzeln zu messen. Für den praktischen Einsatz leitet sich daraus die pragmatischste Empfehlung ab: mindestens Cross-Encoder-Reranking statt MMR einsetzen – der größte Einzelgewinn bei geringstem Umbauaufwand. Innerhalb der Best-of-Breed-Pipeline erweist sich GPT-4o zudem als das mit Abstand stärkste Antwort-LLM ($\varnothing$ 0,863 gegenüber 0,821 für GPT-4o-mini und 0,772 für Mistral); die Produktiv-App bietet Best-of-Breed dementsprechend nur auf dem vollen Produktivkorpus und standardmäßig mit GPT-4o als Antwort-LLM an, trotz dessen höherer Kosten pro Token gegenüber dem Produktiv-Default `gpt-4o-mini`.

Die in Abschnitt 7 genannte Grenze des auf den Demo-Korpus beschränkten Vergleichsumfangs – die auch für das Best-of-Breed-Experiment selbst gilt – schränkt die Übertragbarkeit der genauen Effektgrößen auf den vollen Produktivkorpus ein; eine Validierung des Best-of-Breed-Experiments auf dem vollen Korpus ist bewusst nicht Teil dieser Arbeit. Insgesamt zeigt die Arbeit, dass ein systematischer, austauschbarer Pipeline-Aufbau in Kombination mit einer konsistenten RAGAS-Evaluation es erlaubt, RAG-Designentscheidungen empirisch statt nur intuitiv zu treffen.

7.3 Hinweis zum Einsatz von KI-Werkzeugen

Bei der Erstellung der vorliegenden Arbeit wurde KI-Unterstützung eingesetzt.